

PROJECT TITLE: CLIMATESCOPE VISUALIZING GLOBAL WEATHER TRENDS AND EXTREME EVENTS - GROUP 2

DATA PREPROCESSING

Global Weather Dataset

Presented To
Shanmukha Priya

Presented By
Durga Prasad Singothu

Data Preprocessing

Data Preprocessing is the process of cleaning, transforming, and organizing raw data so that it becomes suitable for analysis, visualization, or machine learning.

Raw data usually contains:

- Missing values
- Duplicate records
- Irrelevant columns
- Inconsistent formats
- Outliers
- Different measurement units

1.Loading the Dataset

Loading a dataset means importing a data file (like CSV, Excel, JSON, etc.) into a programming environment (such as Python) so that we can analyze, clean, and process it.

- ❖ **pandas** → Used to load and handle the dataset (like an Excel sheet in Python).
- ❖ **numpy** → Used for numerical operations.
- ❖ **MinMaxScaler & StandardScaler** → Used later for scaling/normalizing data.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

- ❖ Here, we told Python where our dataset file is located.
- ❖ This path points to the CSV file stored in our system

```
file_path = "/content/GlobalWeatherRepository.csv"
```

- ❖ We used read_csv() to read the CSV file.
- ❖ The dataset is stored inside a variable called df.
- ❖ df is a DataFrame, which is like a table containing rows and columns.

```
df = pd.read_csv(file_path)
print("Dataset Loaded Successfully!\n")
```

2. Inspect Dataset Structure

Inspection of dataset means examining and understanding the basic structure, content, and characteristics of the data before cleaning or analyzing it.

It helps us answer questions like:

- ❖ How many rows and columns are there?
- ❖ What are the column names?
- ❖ What type of data is stored (integer, float, text)?
- ❖ Are there missing values?
- ❖ What are the basic statistics of numerical columns?

❖ What I did

- `df.head()` → Displayed the first 5 rows to see how the data looks.
- `df.info()` → Checked the number of rows, columns, data types, and missing values.
- `df.describe()` → Viewed basic statistics like mean, minimum, and maximum for numerical columns.
- `df.columns` → Listed all column names.

```
# =====  
# 2.. Inspect Dataset Structure  
# =====  
print("First 5 Rows:")  
print(df.head())  
  
print("\nDataset Info:")  
print(df.info())  
  
print("\nStatistical Summary:")  
print(df.describe())  
  
print("\nColumn Names:")  
print(df.columns)
```

3. Identify Missing Values

Identifying missing values means checking the dataset to find empty or null values (NaN) in the data.

Missing values occur when:

- ❖ Data was not recorded
- ❖ There was an error during data collection
- ❖ Some fields were left blank

❖ What I did

df.isnull().sum()

Counted how many empty values are there in each column.

(df.isnull().sum() / len(df)) * 100

Calculated the percentage of missing values.

df.drop_duplicates()

Removed repeated rows from the dataset.

```
# =====  
# 3. Identify Missing Values  
# =====  
print("\nMissing Values Count:")  
print(df.isnull().sum())  
  
print("\nMissing Values Percentage:")  
print((df.isnull().sum() / len(df)) * 100)  
  
df = df.drop_duplicates()
```

4. Removing Columns

Removing columns means deleting the columns that are not needed for our analysis or dashboard.

Sometimes datasets contain:

- ❖ Duplicate unit columns (like Fahrenheit and Celsius both)
- ❖ Technical columns (epoch time, degree values)
- ❖ Extra details not needed for dashboard
- ❖ Very detailed breakdown values

❖ What I did

- Created a list of unnecessary columns.
- Removed those columns from the dataset using `df.drop()`.
- Printed the remaining columns to check the result.

```
# =====  
# 4. Remove Unwanted Columns  
# =====  
unwanted_columns = [  
    'temperature_fahrenheit',  
    'wind_mph',  
    'pressure_in',  
    'precip_in',  
    'feels_like_fahrenheit',  
    'visibility_miles',  
    'gust_mph',  
    'last_updated_epoch',  
    'wind_degree',  
    'timezone',  
    'moonrise',  
    'moonset',  
    'moon_phase',  
    'moon_illumination',  
    'air_quality_Carbon_Monoxide',  
    'air_quality_Ozone',  
    'air_quality_Nitrogen_dioxide',  
    'air_quality_Sulphur_dioxide',  
    'air_quality_PM2.5',  
    'air_quality_PM10',  
    'air_quality_gb-defra-index'  
]  
df.drop(columns=[col for col in unwanted_columns if col in df.columns], inplace=True)  
  
print("\nColumns After Removal:")  
print(df.columns)
```

5. Handling Missing Values

Handling missing values means fixing or filling empty data in the dataset so that it does not cause errors during analysis or dashboard creation.

If we leave missing values:

- ❖ Calculations may fail
- ❖ Graphs may show wrong results
- ❖ Machine learning models may not work

❖ What I did

- Filled missing numerical values (like temperature, humidity) with the median value.
- Filled missing text values (like country, condition) with the most common value.
- Checked again to make sure no missing values remain.
- Converted the date column into proper date format.
- Kept only the latest 6 months of data to reduce size and keep it relevant.

```
# =====  
# 6. Handle Missing Values  
# =====  
  
# Fill numerical columns with median  
num_cols = df.select_dtypes(include=['int64', 'float64']).columns  
df[num_cols] = df[num_cols].fillna(df[num_cols].median())  
  
# Fill categorical columns with mode  
cat_cols = df.select_dtypes(include=['object']).columns  
for col in cat_cols:  
    df[col].fillna(df[col].mode()[0], inplace=True)  
  
print("\nMissing Values After Handling:")  
print(df.isnull().sum())  
  
df['last_updated'] = pd.to_datetime(df['last_updated'])  
  
# Keep only latest 6 months  
df = df[df['last_updated'] > df['last_updated'].max() - pd.DateOffset(months=6)]
```

6. Detecting and Handling Outliers

Outliers are extreme or unusual values in the dataset.

Example:

- ❖ Temperature = 500°C
- ❖ Humidity = -20%

❖ What I did

- Calculated the normal range of values using the IQR method.
- Found values that are too high or too low.
- Replaced those extreme values with safe limit values.
- Kept normal values unchanged.

```
# =====  
# 6. Detect & Handle Outliers (Using IQR Method)  
# =====  
for col in num_cols:  
    Q1 = df[col].quantile(0.25)  
    Q3 = df[col].quantile(0.75)  
    IQR = Q3 - Q1  
  
    lower = Q1 - 1.5 * IQR  
    upper = Q3 + 1.5 * IQR  
  
    df[col] = np.where(df[col] < lower, lower,  
                      np.where(df[col] > upper, upper, df[col]))  
  
print("\nOutliers Handled Using IQR Method")
```


7. Normalizing Numerical Values

Normalization means scaling numerical values into a fixed range, usually between **0** and **1**.

For example:

- ❖ Temperature = 45
- ❖ Humidity = 80
- ❖ Wind Speed = 10

These values are in different ranges.

Normalization converts them into a common scale (0 to 1) so they can be compared properly.

❖ What I did

- Used MinMaxScaler to scale all numerical columns.
- Converted their values into the range of 0 to 1.
- Printed a message to confirm completion.

```
# =====  
# 7. Normalize Numerical Values  
# =====  
scaler = MinMaxScaler()  
df[num_cols] = scaler.fit_transform(df[num_cols])  
  
print("\nNormalization Completed")
```

8. Convert Date Column & Aggregate

- ❖ **Convert Date Column** This means changing the date column into a proper **datetime format** so Python can understand it as a date.
 - ❖ **Aggregate** Aggregation means **grouping data together** and calculating summary values like average, sum, etc.
-

❖ What I did

- Converted the date column into proper date format so Python can understand it.
- Extracted year and month from the date.
- Grouped the data by month.
- Calculated the monthly average of numerical values.
- Displayed the monthly aggregated data.

```
# =====  
# 8. Convert Date Column & Aggregate  
# =====  
# Convert date column (modify column name if needed)  
if 'date' in df.columns:  
    df['date'] = pd.to_datetime(df['date'])  
    df['year_month'] = df['date'].dt.to_period('M')  
  
    monthly_avg = df.groupby('year_month')[num_cols].mean()  
  
    print("\nMonthly Aggregated Data:")  
    print(monthly_avg.head())
```

THANK You